

ITA0612-Machine Learning

CO4 – Industrial Problem Solving

Divyesh S P (192424147)

1.A telecommunications company wants to recommend suitable data plans to customers based on their usage patterns. Design a solution using K Nearest Neighbour or Case-Based Learning. Explain how similarity between users is measured, how recommendations are generated, and how the system adapts to changing user behaviour over time.

Code:

```
# Telecom Data Plan Recommendation using KNN

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# 1. Create customer dataset
data = {
    'Data_GB': [2, 3, 5, 7, 10, 12, 15, 18, 20, 25],
    'Call_Min': [100, 150, 200, 250, 300, 350, 400, 450, 500, 600],
    'SMS': [30, 40, 60, 70, 100, 120, 150, 180, 200, 250],
    'Plan': ['Basic', 'Basic', 'Basic', 'Standard', 'Standard',
            'Standard', 'Standard', 'Premium', 'Premium', 'Premium']
}

df = pd.DataFrame(data)

print("Customer Dataset:")
print(df)

# 2. Select features and target
X = df[['Data_GB', 'Call_Min', 'SMS']]
y = df['Plan']

# 3. Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Get K from user
k = int(input("\nEnter value of K: "))

# 5. Train KNN
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_scaled, y)
```

```

# 6. Get new customer details
data_gb = float(input("Enter Data Usage (GB): "))
call_min = float(input("Enter Call Minutes: "))
sms = float(input("Enter SMS Usage: "))

new_customer = [[data_gb, call_min, sms]]
new_scaled = scaler.transform(new_customer)

# 7. Find nearest customers
distances, indices = knn.kneighbors(new_scaled)

print("\nNearest Customers:")
for i, d in zip(indices[0], distances[0]):
    print(df.iloc[i].to_dict(), "Distance =", round(d, 2))

# 8. Predict recommended plan
prediction = knn.predict(new_scaled)

print("\nRecommended Data Plan:", prediction[0])

# 9. Adapt to new customer behaviour
df.loc[len(df)] = [data_gb, call_min, sms, prediction[0]]

print("\nUpdated Dataset:")
print(df)

```

Output:

```

Enter value of K: 3
Enter Data Usage (GB): 13
Enter Call Minutes: 360
Enter SMS Usage: 130

Nearest Customers:
{'Data_GB': 12, 'Call_Min': 350, 'SMS': 120, 'Plan': 'Standard'} Distance = 0.21
{'Data_GB': 15, 'Call_Min': 400, 'SMS': 150, 'Plan': 'Standard'} Distance = 0.48
{'Data_GB': 10, 'Call_Min': 300, 'SMS': 100, 'Plan': 'Standard'} Distance = 0.71

Recommended Data Plan: Standard

Updated Dataset:

```

	Data_GB	Call_Min	SMS	Plan
0	2.0	100.0	30.0	Basic
1	3.0	150.0	40.0	Basic
2	5.0	200.0	60.0	Basic
3	7.0	250.0	70.0	Standard
4	10.0	300.0	100.0	Standard
5	12.0	350.0	120.0	Standard
6	15.0	400.0	150.0	Standard
7	18.0	450.0	180.0	Premium
8	20.0	500.0	200.0	Premium
9	25.0	600.0	250.0	Premium
10	13.0	360.0	130.0	Standard

2. An energy management company aims to predict electricity consumption based on factors such as time, weather, and appliance usage. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning captures variations in consumption patterns and how the model handles noisy and seasonal data.

Code:

```
# Electricity Consumption Prediction using RBF Network

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Create electricity consumption dataset
data = {
    'Hour': [6, 8, 10, 12, 14, 16, 18, 20, 22, 24,
            6, 8, 10, 12, 14, 16, 18, 20, 22, 24],
    'Temperature': [20, 22, 25, 28, 30, 29, 27, 25, 23, 21,
                  19, 21, 24, 27, 29, 28, 26, 24, 22, 20],
    'Appliance_Usage': [2, 3, 4, 5, 5, 4, 6, 7, 6, 3,
                       2, 3, 4, 5, 6, 5, 7, 8, 6, 3],
    'Consumption': [2.1, 2.8, 3.5, 4.2, 4.5, 4.0, 5.2, 6.0, 5.0, 3.0,
                   2.0, 2.7, 3.4, 4.0, 4.7, 4.2, 5.5, 6.3, 5.1, 2.9]
}

df = pd.DataFrame(data)

print("Electricity Consumption Dataset:\n")
print(df)

# 2. Select input and output
X = df[['Hour', 'Temperature', 'Appliance_Usage']].values
y = df['Consumption'].values

# 3. Normalize input data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Select RBF centers using K-Means
```

```

k = 4
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
kmeans.fit(X_scaled)
centers = kmeans.cluster_centers_

# 5. RBF function
def rbf(X, centers, sigma=1):
    return np.exp(
        -np.sum((X[:, None] - centers) ** 2, axis=2)
        / (2 * sigma ** 2)
    )

# 6. Generate RBF features
Phi = rbf(X_scaled, centers)

# 7. Train output layer
model = LinearRegression()
model.fit(Phi, y)

# 8. Predict consumption
predicted = model.predict(Phi)

# 9. Calculate RMSE
rmse = np.sqrt(mean_squared_error(y, predicted))

# 10. Display results
result = pd.DataFrame({
    'Actual': y,
    'Predicted': np.round(predicted, 2)
})

print("\nActual vs Predicted Consumption:\n")
print(result)

print("\nRMSE:", round(rmse, 3))

# 11. Predict for a new condition
new_data = [[18, 27, 6]]
new_scaled = scaler.transform(new_data)
new_rbf = rbf(new_scaled, centers)
new_prediction = model.predict(new_rbf)

print("\nNew Input:")
print("Hour = 18, Temperature = 27°C, Appliance Usage = 6")

print("Predicted Electricity Consumption:",
      round(new_prediction[0], 2), "kWh")

```

Output:

Actual vs Predicted Consumption:

	Actual	Predicted
0	2.1	2.59
1	2.8	2.56
2	3.5	3.24
3	4.2	4.22
4	4.5	4.09
5	4.0	3.94
6	5.2	5.39
7	6.0	6.06
8	5.0	5.53
9	3.0	2.93
10	2.0	2.63
11	2.7	2.55
12	3.4	3.08
13	4.0	4.24
14	4.7	4.38
15	4.2	4.66
16	5.5	5.66
17	6.3	5.32
18	5.1	5.14
19	2.9	2.89

RMSE: 0.37

New Input:

Hour = 18, Temperature = 27°C, Appliance Usage = 6

Predicted Electricity Consumption: 5.39 kWh